

Technical Architecture & Database Schema Report

AI-Powered Course Recommendation System (Career Compass)

Backend Engineering Team

December 23, 2025

Abstract

This document details the backend architecture and database schema designed for the Career Compass project. The system is built using Django (Python) and PostgreSQL, featuring a normalized database structure specifically tailored to the Kenyan education system (KCSE/KUCCPS). The architecture prioritizes scalability, data integrity, and the complex logic required for cluster point calculation.

Contents

1	Database Schema Design	2
1.1	Module 1: Authentication (App: Users)	2
1.2	Module 2: Student Data (App: Students)	2
1.3	Module 3: Recommendation Logic (App: Universities)	3
2	Project Structure & Implementation	3
2.1	Modular Architecture	3
2.2	Tech Stack	3
3	Progress Report	4
3.1	Completed Tasks	4

1 Database Schema Design

The database is designed around three core logical modules: **Users**, **Students**, and **Universities**. This separation ensures that authentication, user data, and business logic (courses/requirements) remain distinct.

1.1 Module 1: Authentication (App: Users)

We utilize a custom user model to allow for future scalability (e.g., separating Students from Institution Admins).

Table Name	Type	Key Fields & Description
CustomUser	Core	• id (PK) • username, email • is_student (Boolean): Role flag. • is_institution_admin (Boolean): Role flag.

Table 1: User Authentication Schema

1.2 Module 2: Student Data (App: Students)

This module captures the inputs required for the recommendation engine. We prioritized **Normalization** over wide tables. Instead of having 50 columns for different skills, we use a vertical `StudentAttribute` table.

Table Name	Type	Key Fields & Description
StudentProfile	One-to-One	Links to User. Stores <code>kcse_index_number</code> , <code>kcse_year</code> , and <code>mean_grade</code> .
Subject	Lookup	Standard KCSE subjects. Fields: <code>code</code> (e.g., 101), <code>name</code> , <code>category</code> (Group 1-5).
AcademicResult	Transactional	Crucial Table. Maps Student ↔ Subject. Stores the <code>grade</code> (A, B+) and <code>points</code> . <i>Why:</i> Essential for calculating weighted cluster points dynamically.
StudentAttribute	Polymorphic	Consolidates Skills, Interests, and Hobbies into one efficient table. Fields: <code>type</code> (SKILL/INTEREST), <code>name</code> , <code>weight</code> (1-10).
CareerGoal	Data	Stores specific professional targets (e.g., "Software Engineer").

1.3 Module 3: Recommendation Logic (App: Universities)

This module models the KUCCPS structure. It is designed to handle complex prerequisites (e.g., "Requires C+ in Math AND any Group 2 subject").

Table Name	Type	Key Fields & Description
Institution	Entity	Universities/TVETs. Fields: <code>code</code> , <code>name</code> , <code>type</code> (Public/Private), <code>location</code> .
ClusterGroup	Lookup	KUCCPS Clusters (e.g., "Cluster 11: Computing").
Programme	Entity	The actual course. Links to <code>Institution</code> and <code>ClusterGroup</code> . Fields: <code>kuccps_code</code> , <code>job_market_demand</code> .
ProgrammeRequirement	Logic	The Logic Engine. Defines rules like "Subject X $\geq$ C+". Supports both specific subjects and broader categories (e.g., "Any Science").
CutOffPoint	Historical	Stores previous years' weighted cluster points to predict competitiveness.

2 Project Structure & Implementation

2.1 Modular Architecture

We adopted a "Separation of Concerns" philosophy. Instead of a monolithic application, the backend is split into distinct apps located in a clean `apps/` directory.

- **Why this structure?** It allows different team members to work on "Students" and "Universities" independently without merge conflicts. It also makes the codebase easier to test and maintain.

```
1 ai-course-recommender/  
2     manage.py  
3     requirements.txt  
4     course_recommender_backend/           # Project Settings  
5         settings.py  
6         urls.py  
7     apps/                                # Application Modules  
8         users/                           # Custom Auth  
9         students/                       # Student Inputs & KCSE Data  
10        universities/                   # Courses & KUCCPS Logic
```

Listing 1: Directory Structure

2.2 Tech Stack

- **Framework:** Django 5.x (Python) - Chosen for its robust ORM and security features.

- **Database:** PostgreSQL - Chosen for its reliability and support for complex queries needed in recommendation logic.
- **API:** Django Rest Framework (DRF) - For communication with the React Frontend.

3 Progress Report

3.1 Completed Tasks

1. **Environment Setup:** Virtual environment configured, dependencies (`psycopg2`, `drf`) installed.
2. **Database Configuration:** PostgreSQL database `career_compass_db` created and connected.
3. **Schema Implementation:** All models for Users, Students, and Universities have been written and migrated.
4. **Admin Interface:** The Django Admin has been customized with `Inline` views for easier data entry.
5. **Data Seeding:** A custom management command (`seed_data.py`) was created to automatically populate: (`python manage.py seed_data` : this command populates the database with the seed data)
 - All standard KCSE Subjects (Groups 1-5).
 - Top Kenyan Universities.
 - KUCCPS Cluster definitions.